Incompatible API Detection with LLM-Powered Static Analytical Reasoning and Dynamic Simulated Reflection

***Abstract*—Software version updates are frequent and unavoidable as systems evolve to add new features, enhance performance, and address defects. Among these changes, API modifications represent the most prominent and disruptive form of evolution, directly affecting the stability and usability of downstream applications. While additions and removals of APIs are visible, more subtle internal changes—such as parameter adjustments, altered semantics, and modified behaviors—can also introduce significant compatibility issues. Existing approaches for detecting such problems, including static analysis and syntactic differencing techniques, have achieved partial success but remain limited in their ability to capture semantic nuances and generalize across diverse codebases.**

To address these challenges, we present a novel LLM-driven method that leverages the reasoning and code-understanding capabilities of large language models to detect API compatibility issues at both syntactic and semantic levels. Our approach integrates structured change analysis with model-based inference to identify incompatibilities that traditional tools overlook. Through extensive experiments on real-world software projects, we demonstrate that our method outperforms existing baselines, particularly in detecting subtle or complex semantic changes. These results highlight the potential of LLMs to significantly advance the state of the art in API evolution analysis and contribute to more reliable software maintenance.

I. Introduction

Software version updates are a routine and inevitable part of modern software development. As software systems evolve to incorporate new features, improve performance, or fix defects, updates are released frequently across ecosystems. This continuous evolution is essential for maintaining software quality and relevance; however, it also introduces challenges for developers who rely on stable interfaces and predictable behavior. Understanding how version updates propagate through software systems is therefore key to ensuring long-term maintainability. In July 2024, a faulty update to CrowdStrike's Falcon endpoint detection software caused millions of Windows systems worldwide to crash and reboot into blue-screen or recovery mode. This defective update — effectively a broken API/behavior in a widely-deployed component — triggered a global IT outage impacting airlines, hospitals, government services, cloud infrastructure, and more.

Among the various types of changes introduced during software evolution, API modifications represent the most noticeable and impactful updates. APIs serve as the boundary of interaction between components, libraries, and applications. When these interfaces change, downstream systems that depend on them must adapt accordingly. As a result, API updates—whether additions, deletions, or behavioral modifications—often become the primary source of observable change for developers and tools.

Beyond obvious operations such as adding or removing methods, more subtle internal API changes can also significantly affect compatibility. For example, modifications to parameter types, default values, method semantics, or exception behaviors may not be immediately visible but can still break client code or alter expected functionality. Such implicit or semantic changes are particularly difficult for downstream developers to detect, and their resulting incompatibilities may only surface during runtime or testing, often at considerable cost.

Over the years, a variety of techniques have been proposed to detect API compatibility issues, as reflected in prior work such as [**?**], [**?**], [**?**]. These approaches typically rely on static analysis, syntactic differencing, or rule-based models to identify breaking changes. While effective for many classes of incompatibilities, they still face notable limitations. Many tools struggle with semantic reasoning, cannot generalize across heterogeneous codebases, or fail to capture subtle behavioral differences that arise from complex modifications. Consequently, a reliable and generalizable solution remains elusive.

Recent advancements in large language models (LLMs) present new possibilities for addressing this challenge. The strong reasoning, pattern recognition, and code-understanding capabilities of LLMs have been demonstrated in numerous software-evolution-related tasks [**?**], [**?**], [**?**], such as code migration, patch generation, and refactoring analysis. These successes suggest that LLMs are well suited to infer semantic changes, understand API behavior in context, and detect inconsistencies that traditional static tools overlook.

Motivated by these observations, our work introduces a novel method for XXXX. The proposed approach leverages LLM-based reasoning to analyze API changes at both syntactic and semantic levels, enabling a more comprehensive and context-aware detection of compatibility issues. By integrating structured change analysis with model-driven inference, our method aims to bridge the gap between purely syntactic detection mechanisms and deep semantic understanding.

To evaluate the effectiveness of our approach, we conducted extensive experiments using real-world software projects and diverse API evolution scenarios. The results demonstrate that our method effectively identifies a broader range of compatibility issues compared to baseline tools, particularly in cases that involve complex or subtle semantic changes. Quantitative metrics and qualitative analyses both show that LLM-based reasoning significantly enhances detection accuracy and robustness.

In summary, this work makes several key contributions. First, we provide a systematic analysis of the limitations in existing API compatibility detection methods. Second, we introduce a new LLM-driven technique that unifies syntactic differencing with semantic reasoning. Third, we develop an evaluation framework grounded in real-world software evolution data. Finally, our experimental findings offer evidence that LLMs can meaningfully advance the state of the art in API compatibility analysis.

- We
- We
- We

II. Background

A. Python Incompatibility Types

We focus on incompatible APIs in Python, a widely used language whose issues are representative of modern software ecosystems. The incompatible APIs involve the interaction between the upstream dependencies and the downstream client usages. Given an API evolves from an older version to a newer version, the *forward incompatibility* occurs when the newer version introduces changes that prevent existing client code from running correctly, and the *backward incompatibility* occurs when code written for the newer version cannot run correctly on the older version. The incompatible APIs can introduce substantial development overhead and may result in production failures or service disruptions.

Specifically, Python incompatibilities can be categorized into *interface-level changes* and internal-level changes. Interface-level changes occur when supported interfaces are modified, such as changes to API signatures or removed entirely, often leading to immediate failures in client code. In contrast, internal-level changes give rise to *behavioral incompatibilities* that are hidden within API implementations. In such cases, the API signature may remain unchanged, allowing client code to execute without errors, yet the program may exhibit altered semantics due to modifications in default parameters, output values, or exception-handling behavior.

As an interpreted language, Python exhibits many dynamic characteristics, which hinder the early identification of incompatibilities. In many cases, such incompatibilities manifest as *behavioral incompatibilities* that affect program execution without causing immediate failures. These issues are more concealed and difficult to detect at an early stage, and therefore tend to lead to more severe consequences.

B. Real-world Python Incompatibilities

We select three real-world Python incompatibilities to illustrate their hidden characteristics and the practical consequences they can cause.

Example of Return Change. This incompatibility originates from the dictionary iteration APIs (e.g., `dict.items()`, `dict.keys()`) in Python. In Python versions earlier than 3.7, the iteration order of dictionaries was unspecified and could vary across executions, whereas Python $\geq$ 3.7 guarantees that dictionaries preserve insertion order ("*Changed in version 3.7: Dictionary order is guaranteed to be insertion order*" [31], [33]). As a result, client code that implicitly relies on nondeterministic ordering in older versions, or assumes deterministic ordering in newer versions, may exhibit unexpected or incorrect results when executed under the opposite version, despite using the same API and input data.

Example of Exception Change. This incompatibility arises from the generator execution (e.g. `(__iter__())` semantics in Python. Prior to the adoption of PEP 479 [30], raising StopIteration error inside a generator would silently terminate the iteration. After PEP 479, the same behavior results in a raised RuntimeError. Consequently, client code that relies on the generator API without explicitly handling this runtime error may execute correctly in older Python versions but crash at runtime in newer versions, even though the generator interface and invocation remain unchanged.

Example of Default Parameter Change. This incompatibility originates from the `sum()` and `mean()` APIs in NumPy, where the default value of the `dtype` parameter changed across versions. In older versions of NumPy ($\leq$ 1.19), these functions used the same data type as the input array for intermediate computations (e.g., int32) [38], which could result in silent overflow when processing large integer arrays. In newer versions ($\geq$ 1.20), the default parameter was changed to a larger integer type (e.g., int64), preventing overflow [34], [29]. Therefore, client code that relies on the previous behavior may produce different numerical results when executed under a newer version, evqen when the same input and API calls are used.

C. Categories of Incompatibilities

Following the compatibility types presented in Section II-A, we provide a summary of Python incompatibility categories. We base our categorization on the prior studies by Shen et al. [32] and Zhang et al. [47]. Shen et al. identified incompatibilities arising from parameters (CI1-CI5), exceptions (CI6), and return values (CI7). While these studies define seven primary incompatibility types, our labeling process revealed two additional types, which we include in our categorization.

We use Figure 1 as an exemplary code snippet to aid understanding. The figure shows a merge of the code before ($\texttt{API}_b$) and after ($\texttt{API}_a$) the change, where red lines indicate deletions and green lines indicate additions in the later version.

- **Parameter Deletion/Addition (CI1)**: The function parameter is either deleted in the $\texttt{API}_b$ or added in the $\texttt{API}_a$. As shown in Figure 1, `amount` is required in $\texttt{API}_b$ but removed in $\texttt{API}_a$. Calling $\texttt{API}_a$ with (amount=1) will raise "TypeError: fun() got an unexpected keyword argument 'amount'.
- **Mismatched Keys in KWargs (CI2)**: Different keys may exist in the $\texttt{API}_b$ and $\texttt{API}_a$. As shown in Figure 1, $\texttt{API}_b$ extracts the value associated with the key `'key'` from `**kwargs`, whereas $\texttt{API}_a$ retrieves `'key_for_check'`. If the client code invokes $\texttt{API}_a$ using the original `kwargs`, a `KeyError` for `'key_for_check'` may be raised.
- **Parameter Rename (CI3)**: The function parameter from $\texttt{API}_b$ is renamed in the $\texttt{API}_a$. As shown in Figure 1, the parameter `n_size` in $\texttt{API}_b$ is renamed to `n_dim` in $\texttt{API}_a$.

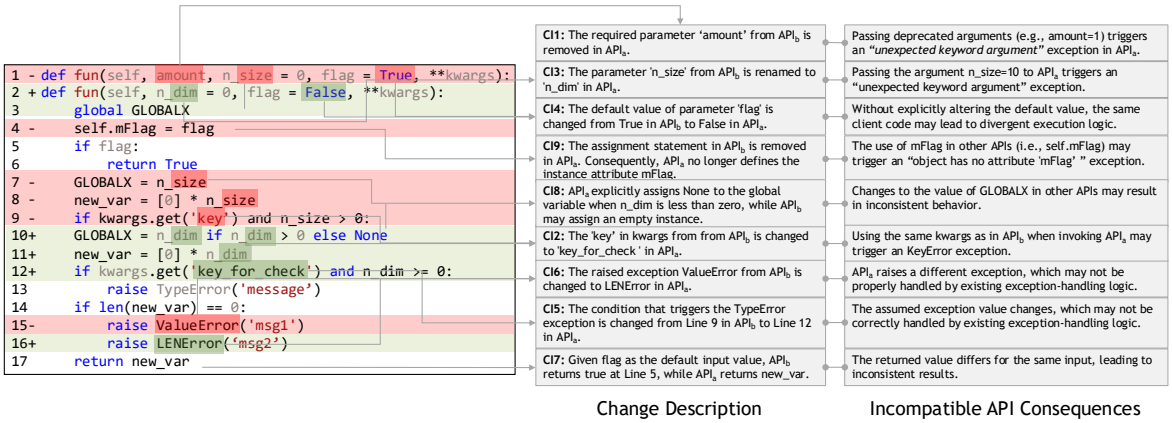

Fig. 1: Illustration of Incompatibility Types

Consequently, invoking $\texttt{API}_a$ with `n_size=value` raises "TypeError: "fun() got an unexpected keyword argument 'n_size'"".

- **Parameter Default Value Change (CI4)**: The default value of a parameter in $\texttt{API}_b$ is modified in $\texttt{API}_a$. As shown in Figure 1, the default value of `flag` changes from `True` in $\texttt{API}_b$ to `False` in $\texttt{API}_a$. Consequently, invoking the API without explicitly specifying `flag` may lead to a different execution path and produce different results. In this example, $\texttt{API}_b$ may return `True` at line 6, whereas $\texttt{API}_a$ continues execution to line 17 and returns `new_val`.
- **Parameter Boundary Value Change (CI5)**: The boundary values of the parameter change across API versions, leading to different behaviors when the same parameter value is supplied. As shown in Figure 1, the condition for raising a `TypeError` changes from `n_size` < 0 in $\texttt{API}_b$ to `n_dim` ≤ 0 in $\texttt{API}_a$. Consequently, providing `n_size` or `n_dim` with a value of 0 yields different outcomes. Specifically, an exception is raised in $\texttt{API}_a$ but not in $\texttt{API}_b$.
- **Exception Change (CI6)**: $\texttt{API}_b$ and $\texttt{API}_a$ raise different exceptions under the same conditions, where one API introduces an exception that is not present in the other and may therefore lack appropriate handling logic. As shown in Figure 1, when `len(new_val)` equals zero, $\texttt{API}_b$ raises a `ValueError`, whereas $\texttt{API}_a$ raises a `LENError`.
- **Return Value Change (CI7)**: The returned value changes given the same client invocation.
  As shown in Figure 1, As noted in CI4, changing a default can cause the API to return different values for the same call (e.g. `False` in $\texttt{API}_{before}$ vs. `new_var(list)` in $\texttt{API}_a$.
- **Global Variable Value Change (CI8)**: The value of a global variable may change even when the same API input is used. As shown in Figure 1, both $\texttt{API}_b$ and $\texttt{API}_a$ reference the global variable `GLOBALX`. When `flag` is `True`, $\texttt{API}_b$ assigns `GLOBALX = n_size`, whereas $\texttt{API}_a$ assigns `GLOBALX = n_dim` or `None`.
- **Instance Attribute Uninitialized (CI9)**: The instance attribute that exists in one version of an API may become uninitialized in another version. As shown in Figure 1, $\texttt{API}b$ assigns `self.mFlag = flag`, whereas $\texttt{API}_a$ does not. Because Python uses a dynamic object model, where instance attributes are created at runtime upon first assignment, `self.mFlag` does not exist in $\texttt{API}_a$.

III. Approach

This section details our approach for detecting Python API incompatibilities. We first outline the approach overview, and then elaborate the core components.

A. Challenges and Design Motivation

Existing static analysis methods (e.g., Shen et al. [32]) rely on handcrafted rules. However, rule design requires substantial expert knowledge, making such approaches difficult to generalize to new libraries or previously unseen incompatibilities. Moreover, rule-based methods fail to capture semantic context. For example, when a parameter is renamed from `arguments` to `args`, the corresponding AST patterns differ syntactically. As a result, rule-based approaches cannot recognize the underlying rename operation.

Having witnessed the effectiveness of large language models (LLMs) in software engineering tasks[10], [11], [43], we naturally think of the opportunities of leveraging LLMs to detect incompatible Python APIs. However, recognizing their limitations and to fully harness their potential, we identify the following three challenges in its practical application.

- **I. Linking incompatibility category description to specific code changes.** Each incompatibility category begins with a brief description of how it manifests. However, when such descriptions are provided directly to an LLM, it may fail to fully infer their correspondence to specific code changes. This misalignment between natural-language descriptions and syntactic code changes creates a gap that can hinder the model's effectiveness.
- **II. Scalability with categories of incompatibilities.** Each incompatibility category exhibits distinct syntactic characteristics. A natural approach is to design a dedicated instruction for the LLM for each category. However, as the number of incompatibility categories grows, this approach would require repeated API scanning and reasoning, leading to excessive token consumption and longer execution times, ultimately rendering it impractical for large-scale analyses.
- **III. LLM hallucination and unreliability.** The LLM may generate hallucinated content and exhibit low confidence

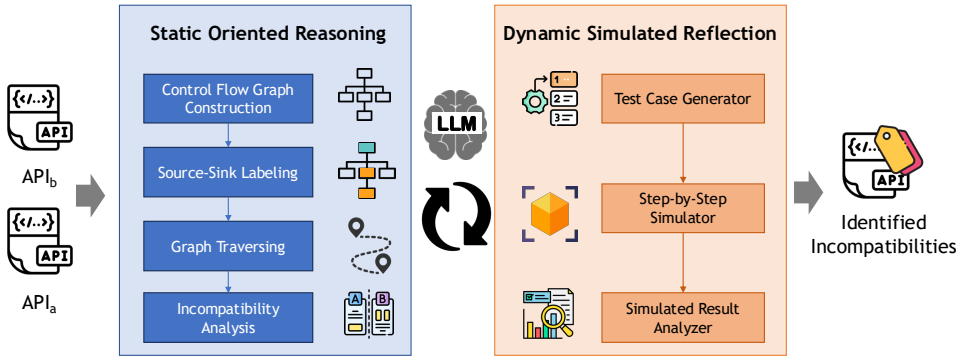

Fig. 2: Overview of COMPACT

in its responses. As a result, some outputs may contain false, contradictory, or inconsistent claims. e.g., citing non-existent code lines to support an assertion.

To address Challenges **I** and **III**, the LLM should fully comprehend the causes of incompatibility from the perspective of code changes and their underlying logic across versions. This requires the LLM to analyze code in a manner similar to a static analyzer, while additionally capturing semantic intent. To tackle Challenge **II**, the LLM should examine the code statements of both versions sequentially from top to bottom and produce intermediate analysis results for each statement. These intermediate states can then be aggregated to determine the final incompatibility category upon completion of the analysis. Furthermore, to mitigate hallucination and unreliability, additional reflection mechanisms are required to validate the LLM's analysis.

Guided by these requirements, we propose decomposing the reasoning process for incompatibility classification into fine-grained steps. The LLM incrementally computes intermediate states while analyzing each statement and integrates them into a unified, structured reasoning framework, thereby addressing Challenges I and II. In addition, we incorporate a self-reflection phase in which the LLM validates its static analysis results from the perspective of test-case behavior.

B. Approach Overview

We propose COMPACT. As shown in Figure 2, COMPACT is divided into two phases: *Static-oriented Incompatibility Reasoning* and *Dynamic Simulated Reflection*.

In the first phase, COMPACT performs control-flow graph (CFG) construction, source-sink labeling, graph traversal, and incompatibility analysis. Both API_b and API_a are analyzed once during CFG construction. Subsequently, source-sink labeling and graph traversal are conducted on the constructed CFGs. The notions of source and sink originate from security analysisCITE. In our context, sources correspond to input variables (e.g., parameters), while sinks denote statement endpoints that may trigger incompatibilities. COMPACT then traverses the CFGs to collect statements along paths from sources to sinks. Finally, all incompatibility categories are determined in a single incompatibility analysis step by comparing the source-sink paths and their intermediate data objects. We structure incompatibility reasoning to align LLM understanding with code-level changes, thus addressing Challenge I, and design a workflow that supports multiple incompatibility types, thus addressing Challenge II.

Prompt for Control Flow Graph Construction

You are a static program analysis engine tasked with constructing a Control Flow Graph (CFG) for the given Python source code.
Requirements
1. Nodes: Each node represents at most one statement or a expression-level execution unit, with a single entry point, and a single exit point.
2. Edges: The directed edges represent possible control transfers between nodes. Explicitly label edges with conditions (e.g., if condition=True/False), and include fall-through edges, loop back-edges, exceptional edges.
3. Language Constructs: The CFG construction should handle Python language constructs, including but not limited to conditionals, loops, function-level control flow, exception handling, and context managers.
4. Entry and Exit Nodes. Create a single Entry node representing function start, and a single **Exit** node representing normal termination. All return statements must connect to the Exit node.
Now you will be given input as a function, output the CFG in the following format:

Fig. 3: Prompt for Control Flow Graph Construction

In the second phase, COMPACT comprises a test case generator, a step-by-step simulator, and a simulated result analyzer. Unlike the first phase, this phase reflects on the identified incompatibility categories and aims to generate valid test cases that validate the correctness of the determined categories. The process simulates a dynamic execution scenario in which the LLM generates test cases, infers intermediate results as statements are executed, and ultimately confirms the incompatibility category based on these intermediate outcomes. This phase addresses Challenge III.

C. Static-oriented Incompatibility Reasoning

This section details our static analysis framework. Unlike traditional compiler-level approaches, we employ a lightweight, semantic abstraction designed to efficiently detect behavioral incompatibilities between API versions.

1) Control Flow Graph Construction: We formulate Control Flow Graph (CFG) construction as a structured inference task performed by an LLM guided by an explicit static-analysis prompt. The CFG is constructed in a single pass over the function body. This one-pass design is critical for scalability, as it supports efficient analysis across multiple incompatibility types. We provide the prompt in Figure 3. Given a Python function as input, the LLM is instructed to reason about program execution syntax and infer control-flow structure at the function level. The model first introduces a single Entry node representing function invocation and a single Exit node representing normal termination. The function body is then decomposed into fine-grained execution units, where each

TABLE I: Categories of API Incompatibilities and Reasoning Process

Id.	Name	Reasoning Process
CI1/CI3	Parameter Addition/Deletion and Rename	Parameters are extracted from the pre- and post-change API signatures and first matched by name to identify unchanged parameters. Unmatched parameters are candidates for addition, deletion, or renaming. Since parameter renaming preserves data-flow traces while true additions or deletions do not, we distinguish these cases by comparing their data-flow traces across versions.
CI4	Parameter Default Value Change	Building on the detection logic of CI1 and CI3, we identify CI4 by comparing the default values of mapped default-valued parameters. When both parameter names and default values change, we first resolve parameter renaming using CI3, and subsequently compare the mapped default values to determine CI4.
CI2	Kwargs Key Deletion/Addition	Keyword-argument keys are accessed through APIs such as `kwargs.get('key')`. We identify key additions or deletions by statically analyzing `kwargs` usages in the function body and comparing the sets of accessed keys between API versions.
CI5	Parameter Boundary Range	Branch conditions are extracted and mapped across pre- and post- change APIs. For each mapped condition, we trace data flow to input parameters and compare boundary constraints, and different boundaries indicate a CI5 incompatibility.
CI6	Exception Addition/Deletion/Change	First, we should locate raised exceptions in both the pre-change and post-change APIs and trace the data flow from input parameters to each exception. Exceptions are mapped across versions based on the similarity of their data-flow paths and control conditions. Unmapped exceptions are classified as exception additions or deletions. For mapped exceptions, differences in input-parameter condition constraints indicate an exception incompatibility.
CI7	Return Value Change	We extract all return statements from the pre- and post-change APIs and compare their predecessor structures. Return predecessors are mapped across versions based on the semantic similarity of their control conditions and returned expressions. Unmapped predecessors indicate a return type change, while for mapped predecessors, differences in return conditions, value constraints, or controlling predicates are reported as a return value change.
CI8	Global Variable Change	We identify assignments to global variables in both API versions and group them by variable name. For variables appearing in both versions, we compare the control-flow conditions under which the variables are assigned. Differences in guarding conditions or value constraints indicate a global variable change.
CI9	Instance Attribute Uninitialized	We identify assignments to instance attributes in both API versions and compare their presence across versions. If an attribute is assigned in one version but has no corresponding assignment in the other, we report an instance attribute uninitialized incompatibility.

CFG node corresponds to at most one statement or expression, ensuring a single entry and exit per node.

The LLM infers directed edges between nodes to represent all feasible control transfers, including fall-through edges, conditional branches with explicitly labeled true/false outcomes, loop back-edges, and exceptional control-flow edges arising from exception handling and context managers (Requirement 3). To preserve precise control-flow semantics, all return statements are connected directly to the Exit node (Requirement 4). Each inferred edge is annotated with the corresponding control condition and source-level information (e.g., line numbers), producing an explicit and machine-readable representation of execution paths.

The constructed Control Flow Graph is represented as a list of directed edges. Each edge is encoded as a tuple containing the source node identifier (*from_id*) and its corresponding code content (*from_content*), the destination node identifier (*to_id*) and its content (*to_content*), and an edge label (edge) that describes the control-flow semantics. The CFG is expected to adhere to the following format:

```
[{
  "from_id": ID, "from_content": content,
  "to_id": ID, "to_content": content,
  "edge": "If, Line Number, True"
 }...]
```

The edge label explicitly records the control construct type (e.g., conditional branch), the source line number, and the branch (e.g., True or False). This representation captures both structural and semantic control-flow information while remaining compact and machine-readable.

Prompt for Source-Sink Labeling

You are an expert Python developer with expertise in control-flow analysis. You will be provided with a Control Flow Graph (CFG) constructed from a Python function. Your task is to traverse the CFG from the Entry node and identify source and sink nodes according to the definitions below.
The analyzed function has the following signature: *Foo(bar)*
A source is the first CFG node along its path that references one of its function parameters.
A sink can either be:
1. The predecessor of the exit node. E.g., a node that contains a return statement.
2. The CFG node along its path that performs a last assignment to a global variable or a class attribute.
3. A node that contains a raise statement.
The CFG is given as follows:
CFG CONTENT
Output the source/sink node in the following format:

Fig. 4: Prompt for Source-Sink Labeling

2) Source-Sink Labeling: Given a Control Flow Graph constructed for a Python function, we perform source-sink labeling by traversing the CFG starting from the Entry node. This process is framed as a structured reasoning task executed by an LLM with expertise in control-flow analysis. The traversal follows feasible execution paths and inspects each visited node to identify source and sink semantics based on predefined rules. We provide the corresponding prompt in Figure 4.

Specifically, a source is defined as the first CFG node encountered along an execution path that references a function parameter. This definition ensures that each path has at most one source node and captures the earliest point at which external input enters the control flow. The sources correspond to the function parameters, including positional arguments, optional parameters with default values, and `**kwargs`, collectively defining the input surface of the API. While `**kwargs` itself is labeled as a source, we treat each individual key accessed via

Prompt for Graph Traversing

You are an expert Python developer with expertise in control-flow analysis. You will be provided with a Control Flow Graph (CFG) and identified source and sink nodes. Your task is to traverse the CFG from the source and sink nodes and identify source-sink paths. The analyzed function has the following signature: *Foo(bar).* The CFG is given as follows: # CFG CONTENT. The source and sink nodes are: # SOURCE and SINK nodes. Now you will output the source-sink paths in the following format:

Fig. 5: Prompt for Graph Traversing

`kwargs.get('key')` or `kwargs['key']` as a distinct, fine-grained source. These keys are tracked separately in subsequent analyses to capture precise parameter-level data flow.

A sink represents a terminal or side-effect-inducing operation in the context of incompatibilities. We identified sinks under three conditions: (1) *exit_pred*: a node that precedes the Exit node, such as a return statement; (2) *global_var* or *attri_var*: the last assignment along a path to a global variable or a class attribute; or (3) *raise*: a node that contains a raise statement. Each identified source or sink is emitted as a structured record containing its classification (`source` or `sink`), an explicit sink subtype for sinks, and the corresponding CFG node id and its content. The result is expected to adhere to the following format:

```
[{
 "type": "source|sink",
 "sink_type": "exit_pred|global_var
              |attri_var|raise"
 "node": ["node_id": ID,
             "node_content": content]
}]
```

3) Graph Traversing: Given a Control Flow Graph and a set of identified source and sink nodes, we extract source-sink paths by traversing the CFG from each source to its corresponding sinks. The traversal explores all feasible execution paths, including normal flows, loops, conditional branches, and exceptional edges, ensuring that all paths from inputs to outputs or side-effect locations are captured. We provide the corresponding prompt in Figure 5.

Specifically, each source-sink path is represented as an ordered sequence of CFG edges. Each edge records its source and destination node IDs and content, along with the associated control-flow condition (e.g., the outcome of an `if` statement and the source line number). The output is structured as a list of paths, each assigned a unique `path_id` and labeled with the corresponding source-sink pair. Each path includes the sequence of edges in the format:

```
[{
 "path_id": 1,
 "source-sink": "",
 "edges":[{
   "from_id": ID, "from_content": content
    "to_id": ID, "to_content": content,
   "edge": "If, Line Number, True"
   }, {
   "from_id": ID, "from_content": content
    "to_id": ID, "to_content": content,
   "edge": "If, Line Number, True", ...
   }]
}]
```

Prompt for Incompatibility Analysis

You are an expert Python developer with expertise in control-flow analysis. You will be provided with a Control Flow Graph (CFG) and identified source-sink paths of two Python API versions (API_b and API_a). Your task is to traverse the CFG paths and analyze the incompatibilities between the two versions. Follow the detection logic step-by-step:

1. Parameter Incompatibilities

Group the source–sink paths by parameter (source) name in the form <source, <source–sink path>>. Then perform: *(1) Default Parameter Value Change Analysis (CI4).* Compare the parameter (source) names between API_b and API_a. For parameters that exist in both versions, identify the parameter default value change (CI4) if the parameter has the same name but different default values across the two versions. *(2) Parameter Addition, Deletion and Rename Analysis (CI1, CI3).* Identify parameters that are added in API_a or deleted from API_b. For each added parameter in API_a, compare it against each deleted parameter from API_b by comparing their source–sink paths: If the source–sink paths are semantically similar, identify the change as a Parameter Rename (CI3); Otherwise and the added parameter is not an optional parameter, classify the change as a Parameter Addition (CI1). The source-sink paths are semantically similar if the statements within the paths exhibit semantic equivalence. The same applies to API_b. *(3) Kwargs Key Addition, Deletion Analysis (CI2). E*xtract the keys accessed along their source–sink paths in both API_b and API_a (e.g., via kwargs.get(key) or kwargs[key]). If the accessed keys differ between the two versions, identify it as a Kwargs Key Addition/Deletion (CI2).

2. Exceptions Incompatibilities

Select the source–sink paths whose sink_type is raise, and group them by exception (sink) name in the form <sink, <source–sink path>>. Then perform: *(1) Exception Addition or Deletion (CI6).* Compare the exception (sink) names between API_b and API_a. If an exception name appears in one version but has no corresponding exception in the other version, identify the change as an Exception Change (CI6) (addition/deletion). *(2) Parameter Boundary Range Change (CI5).* For exception names that appear in both API_b and API_a, compare their corresponding source–sink paths. Extract the control-flow conditions under which the exception is raised (e.g., conditional expressions guarding the raise statement) , and synthesize the input-parameter (source) value ranges required to trigger each exception. If the exception is raised under different parameter value constraints between the two versions, report the change as a Parameter Range Change for Exception (CI5).

3. Non-local Variable Incompatibilities

Select the source–sink paths whose sink_type is global_var or attri_var, and group them by variable (sink) name in the form <sink, <source–sink paths>>. Then, for variables that appear in both API_b and API_a, compare their source–sink paths. Extract the control-flow conditions under which the variable is assigned (e.g., guarding predicates or value constraints). If the global variable is assigned under different conditions, report the change as a Global Variable Value Change (CI8). If the attribute variable is assigned in one version but unassigned in the other, report the change as an Instance Attribute Unitialized (CI9).

4. Return Incompatibilities

Select the source–sink paths whose sink_type is exit_pred. First, compare the number of predecessors (i.e., <predecessor, <source–sink path>> pairs) between API_b and API_a. Then perform predecessor mapping across versions. For each <predecessor, <source–sink path>> pair in API_b and API_a, identify corresponding predecessors across the two versions by matching them based on the semantic similarity of their source–sink paths. Identify predecessors that cannot be mapped between API_b and API_a. If a new predecessor type appears in either version, report the change as a Return Type Change. For each mapped predecessor pair, compare their corresponding source–sink paths. If the return conditions, value constraints, or controlling predicates differ between the two versions, report the change as a Return Value Change (CI7).

Fig. 6: Prompt for Incompatibility Analysis

4) Incompatibility Analysis: To systematically detect incompatibilities between two Python API versions, we leverage a control-flow-based analysis on source-sink paths extracted from the function's Control Flow Graph. The analysis is organized into four categories: parameter, exception, global variable and class attribute, and return incompatibilities. We provide the corresponding prompt in Figure 6.

1. Parameter Incompatibilities. Parameter-level changes can significantly impact the behavior of an API. We first group source-sink paths by parameter name and perform a stepwise comparison between the two API versions. Default value changes (CI4) are identified when a parameter exists in both versions but its default value differs. Such changes may silently affect function behavior if callers rely on defaults. Parameter addition, deletion, and renaming (CI1, CI3) are detected by comparing added and removed parameters along with the semantic similarity of their source-sink paths. If the paths are semantically equivalent, an added parameter is considered a renamed parameter; otherwise, it is classified as a new addition. This ensures that structural changes in the API interface are accurately captured. Kwargs key changes (CI2) are analyzed by inspecting keys accessed through kwargs in both versions.

If the set of keys differs, we classify the discrepancy as a Kwargs Key Addition or Deletion, reflecting modifications in the API's flexible keyword arguments.

2. Exception Incompatibilities. Changes in exception handling can break client code or change the conditions under which errors are triggered. We isolate source-sink paths where the sink represents a raise statement and group them by exception type. Exception addition or deletion (CI6) is detected when an exception appears in one version but not in the other. Parameter boundary changes (CI5) are captured by analyzing the control-flow conditions guarding each raise statement. If the triggering parameter value ranges differ between API versions, we report a Parameter Range Change for Exception, highlighting changes in the preconditions required to raise an exception.

3. Non-local Variable Incompatibilities. APIs may modify global or instance-level variables, and changes in these assignments can propagate unexpected side effects. We examine source-sink paths whose sink corresponds to a global or attribute variable. For global variables, differences in the control-flow conditions under which the variable is assigned are reported as a Global Variable Value Change (CI8). For instance attributes, if a variable is assigned in one version but unassigned in the other, it is marked as an Instance Attribute Uninitialized change (CI9). This captures cases where object state initialization diverges across versions.

4. Return Incompatibilities. Changes in the return behavior of an API can directly affect downstream computations. We focus on source-sink paths with a exit_pred sink, comparing the number of predecessors and mapping them across versions using semantic similarity. Return type changes are identified when a new predecessor type appears in one version that has no corresponding predecessor in the other. Return value changes (CI7) are reported when mapped predecessors exhibit differences in return conditions, value constraints, or controlling predicates. This ensures that modifications in the API's output behavior are systematically captured.

Overall, this multi-step analysis leverages semantic equivalence and control-flow reasoning to provide a precise, interpretable identification of API incompatibilities. By considering parameters, exceptions, non-local variables, and return behavior, the method can uncover both overt and subtle changes that may affect client code.

D. Dynamic Simulated Reflection

To address judgment biases in the incompatibility category candidates generated by the CoT-LLM, we design a multi-agent collaborative validation framework. It implements accurate verification of candidate categories through a process of "test case generation, simulation execution, and comprehensive evaluation," ultimately filtering out truly valid API incompatibility types. The framework comprises three functionally independent yet collaboratively working agent modules: the Test Case Generation Agent, Simulation Agent, and Comprehensive Analysis Agent. These modules form a validation chain through explicit information interaction, ensuring the objectivity and reliability of validation results.

1) Test Case Generation: The Test Case Generation Agent serves as the initiator of the validation process, tasked with constructing targeted test inputs to probe potential incompatibilities. Its core functionality is to generate pairs of calling statements—one for the old API version and one for the new—along with supplementary context, based on the provided old and new API code snippets, the candidate incompatibility category, and the underlying reasoning. The design of these test cases adheres to a critical principle: they must be highly likely to trigger the alleged incompatibility if it indeed exists. For instance, when evaluating a candidate issue related to changed default parameter values, the agent will craft calling statements that omit the affected parameters, thereby exposing discrepancies in behavior caused by the default value shift. The output of this agent follows a structured format, explicitly separating the calling statements and supplementary information for the old and new APIs to ensure clarity in subsequent processing.

The collaborative workflow of the three agents forms a closed-loop validation pipeline. Starting with a candidate incompatibility category, the Test Case Generation Agent tailors probes to expose potential issues, the Simulation Agent generates empirical behavioral data, and the Comprehensive Analysis Agent renders a data-driven judgment. This division of labor ensures that each stage of validation is handled by a specialized component, reducing the risk of subjective biases that might arise from a single model's judgment. By structuring the validation as an explicit information flow—from test design to execution to analysis—the framework enhances the transparency and reproducibility of the verification process. Ultimately, this multi-agent approach strengthens the reliability of incompatibility category identification, providing a robust mechanism to filter out spurious candidates and retain only those that are empirically validated.

2) Step-by-step Simulation: Upon receiving the test cases from the Test Case Generation Agent, the Simulation Agent takes on the role of executing these tests in a controlled environment. Its primary responsibility is to simulate the runtime behavior of both the old and new API versions when invoked with the generated test inputs. This involves parsing the API code, instantiating necessary objects (if applicable), and executing the calling statements step-by-step. The agent meticulously records the execution results: if the program exits normally with a return value, it captures the specific output; if an exception is raised (e.g., missing arguments, type errors), it documents the error type and description. By faithfully replicating runtime scenarios, this agent transforms abstract code differences into concrete behavioral outcomes, providing empirical data for validating the alleged incompatibility.

3) Simulated Result Analysis: The Comprehensive Analysis Agent acts as the final evaluator, synthesizing information from the preceding stages to assess the validity of the candidate incompatibility category. It takes as input the definition of the incompatibility type, the reasoning behind the initial diagnosis, and the behavioral results generated by the Simulation Agent. Using a scoring mechanism ranging from 0 to 10, the agent evaluates the alignment between the diagnosed incompatibility, the test case design, and the observed runtime

behaviors. The score is determined based on three key criteria: logical consistency (whether the reasoning logically leads to the alleged incompatibility), rigor of evidence (whether the simulation results clearly demonstrate the expected behavioral discrepancy), and coherence (whether all components of the validation process reinforce the same conclusion). A higher score indicates greater confidence in the existence of the specific incompatibility, while a lower score suggests either flaws in the initial diagnosis or inadequacies in the test scenario.

IV. Evaluation

Our experiments are designed to answer the following research questions:

- **Accuracy Evaluation (RQ1)**: How is the overall accuracy of CompAct compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2)**: How is the effectiveness of CompAct in different API compatibility types?
- **LLM Generalizability (RQ3)**: How is the accuracy of CompAct using different LLMs?
- **Ablation Study (RQ4)**:

A. Dataset

The Python third-party library Application Programming Interface (API) compatibility issue dataset constructed in this study comprises two core components: a benchmark dataset derived from existing research, which includes 108 incompatible API pairs built by Shen et al. on the Flask and pandas libraries; and an extended dataset supplementary constructed in this work. Through systematic repository screening, Issue/Pull Request (PR) analysis, and multi-round validation, the extended dataset adds over 70 valid samples, resulting in a total of more than 130 compatibility issue annotations. The dataset comprehensively covers third-party libraries across multiple application domains, such as web development, data analysis, and cloud services, and includes API compatibility issues across major, minor, and patch version transitions. This design provides multi-scenario and fine-grained experimental data support for research on Python third-party library API compatibility detection.

Dataset A.

To construct the dataset of 108 incompatible API pairs, Shen et al. selected six version pairs from two representative Python third-party libraries (Flask for web development and pandas for data analysis), covering cross-major, cross-minor, and cross-patch version transitions. A combined approach of regression testing and version update log analysis was adopted to ensure the comprehensiveness and reliability of the dataset. For regression testing, test cases from the previous version (v1) of each library were executed on the updated version (v2); failed test cases indicated potential incompatible APIs, leading to the identification of 70 incompatible API pairs from 58 failed test files among 2,290 regression test files across the six version pairs. Complementarily, version update logs (including intermediate logs for cross-patch version pairs to capture persistent changes) were analyzed to identify additional incompatible APIs by targeting keywords such as ”parameter reduction”, ”added processing”, and ”no longer supported”, resulting in 38 supplementary incompatible API pairs from 11 analyzed logs. To enhance data credibility, the collection process was independently conducted by the first two authors, with discrepancies resolved through group discussions organized by the third author, involving joint reviews of source code, regression test results, and update logs to confirm the existence of compatibility issues for controversial API pairs. This dual-method strategy effectively mitigated limitations associated with incomplete test coverage in unit testing and inconsistent log quality, ultimately forming a dataset of 108 incompatible API pairs that served as the foundation for subsequent empirical analysis.

Dataset B.

We adopted a different strategy to construct our own dataset to cover codes from more areas. We first selected the top 10 Python libraries by download volume over the past 30 days (as of May 1, 2025) from ”https://hugovk.github.io/top-pypi-packages/top-pypi-packages-30-days.json”. Those libraries are the most popular ones and can represent most python code usage. However, to cover most compatibility issues, for each candidate repository, we also selected additional relevant Python repositories because some issues are not directly reported in the source repository but in relevant Python repositories.

In the preliminary screening of Issues and PRs, for each selected repository, we cycled through a list of keywords (including ’compatibility’, ’compatible’, ’version’, ’evolution’, ’exception’, ’TypeError’, ’AttributeError’, ’ImportError’, ’breaking changes’) and used the GitHub Issues Search API (/search/issues) to retrieve issues and PRs matching these keywords.

Thereafter, approximately 400 entries were filtered through two rounds of LLM-based checks, followed by manual review. In the LLM analysis phase, the LLM was employed to extract structured information from the issues, including username, repository, version tag1, version tag2, file path, object name, and llm analyse compatibility. Multiple GitHub checks were conducted on the extracted results to ensure the validity of the fields. Specifically, the first LLM was used to convert raw issues into structured compatibility problem entries, which constituted the core information extraction of the dataset, extracting key information such as error type, root cause, library, version, affected files, breaking change, and recommended version. Additionally, the second LLMs was used to supplement and explain the technical details of compatibility issues, especially the causal chain between API changes and errors, serving as an in-depth evidence supplement of the dataset.

After filtering of LLMs, from the original 2000 samples, xxx was ...

Finally, manual verification was performed to confirm the existence of incompatibilities and the correctness of Python API paths, followed by annotation. Ultimately, over 70 samples were obtained, with a total of more than 130 annotations added.

B. RQ Setup

To study RQ1, we...

Baselines. We selected XXX , XXX, and XXX as the comparison.

TABLE II: Top Python Libraries by Download Volume

Library	Category	relevant project counts
boto3	Web development	9
urllib3	Web development	7
requests	Web development	10
botocore	Web development	6
certifi	Security	3
charset	xxx	0
normalizer	xxx	0
idna	xxx	3
setuptools	xxx	10
aiobotocore	xxx	3
python-dateutil	xxx	2

TABLE III: Evaluation Result Compared with SOTA

Method	Publication	Accuracy	Subset Acc.	Macro-Pre.	Micro-Pre.	Macro-Rec.	Micro-Rec.
Statistic Baseline Model	Shen et al.	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
PYCOMPAT	Zhang et al.	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
PCART	Zhang et al.	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
COMPACT (Ours)	-	**0.xxx**	**0.xxx**	**0.xxx**	**0.xxx**	**0.xxx**	**0.xxx**

TABLE IV: Evaluation Result Compared in Different Compatibility Type

No.	Incompatibility Category	Incom. API (#)	TP	FP	FN	Pre.	Rec.
CI1	Param Add or Delete	xx	xx	xx	xx	0.xxx	0.xxx
CI2	Key of Keyword Argument Add or Delete	xx	xx	xx	xx	0.xxx	0.xxx
CI3	Param Rename	xx	xx	xx	xx	0.xxx	0.xxx
CI4	Param Default Value Change	xx	xx	xx	xx	0.xxx	0.xxx
CI5	Param Range Change	xx	xx	xx	xx	0.xxx	0.xxx
CI6	Throw Different Exception	xx	xx	xx	xx	0.xxx	0.xxx
CI7	Return Value Change	xx	xx	xx	xx	0.xxx	0.xxx
CI8	Global Variable Change	xx	xx	xx	xx	0.xxx	0.xxx
CI9	Reference Parameter Change	xx	xx	xx	xx	0.xxx	0.xxx
	Macro-Average	xx	xx	xx	xx	0.xxx	0.xxx

TABLE V: Evaluation Result for Different LLMs

LLMs	Accuracy	Subset Acc.	Macro-Pre.	Micro-Pre.	Macro-Rec.	Micro-Rec.
deepseek-chat	**0.xxx**	**0.xxx**	**0.xxx**	**0.xxx**	**0.xxx**	**0.xxx**
deepseek-reasoner	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
qwen-plus	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
gpt-3.5-turbo	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
gemini-2.5-flash	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
claude-3-5-haiku-20241022	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx

TABLE VI: Evaluation Result for Different Method

Evaluation	binary(COT)	ours(COT)	ours(one-pass-COT)	ours(one-pass-COT+agent)	ours(one-pass-COT+agent+example)
All-Precision					
All-Recall					
CI1-P					
CI2-P					
CI3-P					
CI4-P					
CI5-P					
CI6-P					
CI7-P					
CI9-P					

Metrics. We use precision and recall as metrics, which are defined as follows:

C. Results

1) Accuracy Evaluation (RQ1):

2) RQ2:

3) RQ3:

4) RQ4:

V. Related Work

A. API Incompatibility Detection

The API compatibility comes with the subsequence of API evolution, which has been extensively studied. Lamothe et al. [17] conducted in their systematic API evolution review that automatic identifying API change causes and uniform benchmarks as the main remaining challenges. In the context of API compatibility, Cai et al.[3] conducted a large-scale longitudinal study on Android runtime compatibility, while Liu et al.[22] summarized five categories of API-induced compatibility issues. More recently, the focus has deepened into device-specific compatibility behaviors [7] and compatibility issues caused by vendor customizations [4]. Beyond mobile, such challenges are also prevalent in system software like Linux [36], programming languages and its client applications like Java [13] and NPM ecosystem [15]. In deep learning systems, compatibility issues spanning software and hardware stacks have been rigorously characterized [40].

To tackle the problem of incompatibility detection, a natural approach is to construct and execute test cases to trigger API behaviors across different versions. Sun et al. [35] proposed JUnitTestGen, which mines real-world API usages to generate executable unit tests. Going a step further to ensure comprehensive coverage, Mahmud et al. [25] introduced APICIA, which performs a bidirectional impact analysis. Besides, to address client-specific upgrades, Zhu et al. [49] utilized a knowledge-guided framework to generate incompatibility-revealing tests, while Chen et al. [5] leveraged cross-project testing to detect behavioral backward incompatibilities in Python. Beyond standard testing, more sophisticated dynamic techniques have been introduced to analyze complex dependency chains. Kong et al. [16] employed dynamic object relation graphs to precisely detect breaking changes in JavaScript. Furthermore, Cheng et al. propose ReadPyE [6], which addresses the challenge that dynamic analysis requires valid execution environments by automatically generating compatible Python runtimes.

However, testing-based analysis is often limited by high computational costs, its reliance on existing test suites, and inherent coverage issues. In contrast, static analysis offers a more scalable and practical alternative. Addressing device fragmentation, Wei et al. [41] pioneered FicFinder to characterize device-specific issues, a direction later advanced by Pivot [42] to learn API-device correlations from large corpora. To tackle general API evolution, Li et al. [20] introduced CiD to model API lifecycles. He et al. [9] developed IctApiFinder to detect incompatible usages. Beyond general compatibility issues, existing work has examined several specific types of problems, including complex callbacks [26], [12], field-

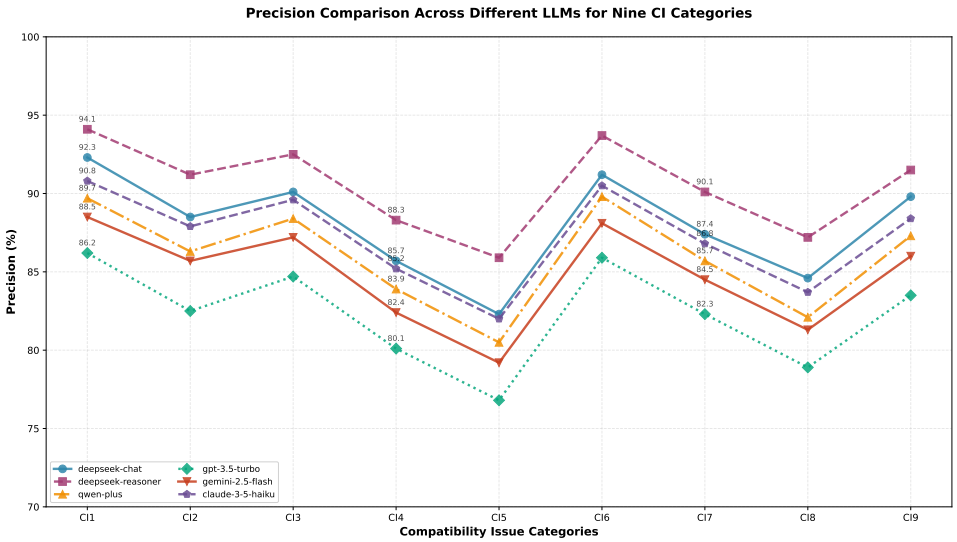

Fig. 7: Precision of Different LLMs in Each Compatibility Type

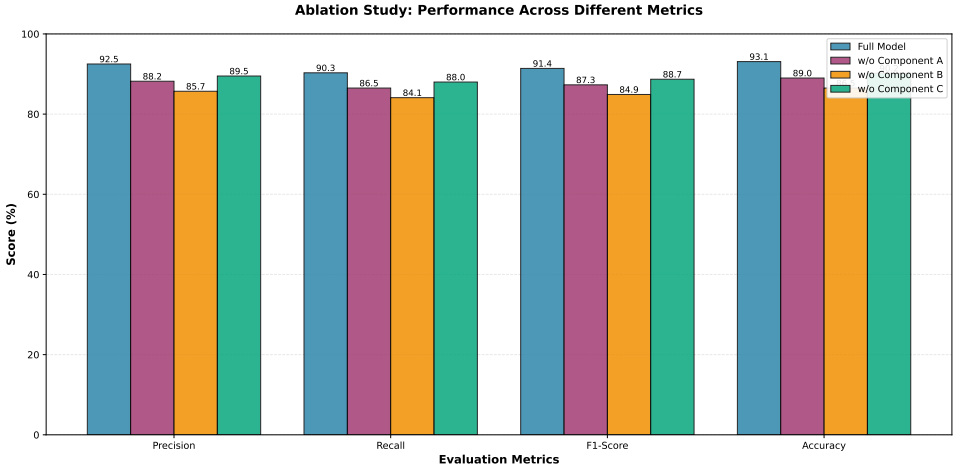

Fig. 8: Evaluation of Different Methods

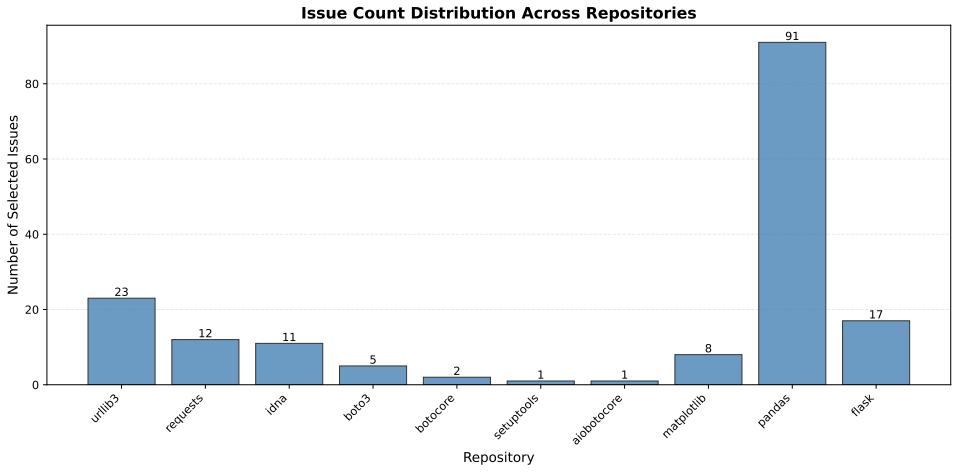

Fig. 9: Dataset Distribution of Libraries and Issues

level evolutions [27], AndroMevol-related changes [23], and incompatibilities caused by recurring design smells [14]. In Java, APIDiff [2] and Roseau [18] focus on identifying breaking changes. In Python, static analysis has been tailored to its specific ecosystem. Zhang et al.[47] developed PyCompat to handle framework API evolution, while Zhao et al. [48] constructed knowledge bases to detect version incompatibilities in deep learning stacks. Other studies target specific risks such as breaking changes in packages [8], deprecated APIs [37], and complex variadic parameters [45]. More recently, static

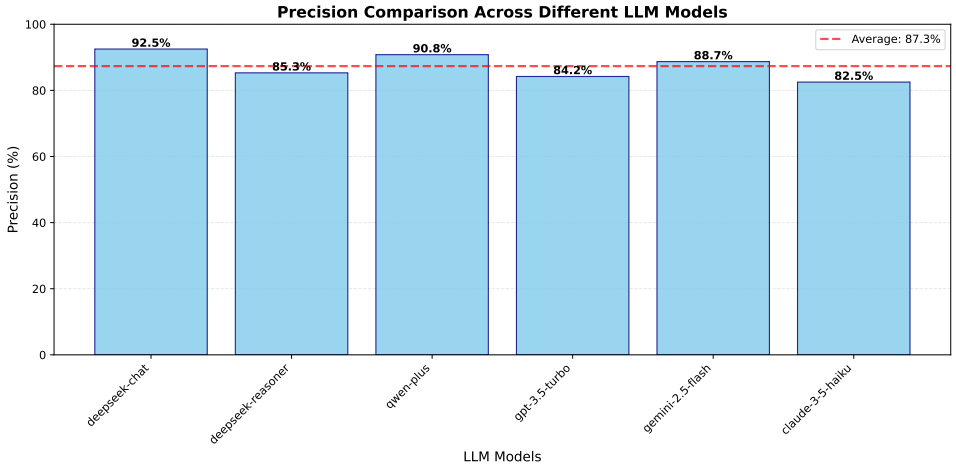

Fig. 10: Precision of Different LLMs

approaches have expanded to classify Python incompatibility types [32] and solve cross-repository issues via offline database construction [44].

Despite the advancements, the static-based approaches encounter severe bottlenecks. State what bottlenecks?. To bridge these gaps, we integrate LLMs, enhanced via Chain-of-Thought (CoT) reasoning and multi-agent collaboration to achieve accurate XX.

B. LLMs for Library Evolution

With the rapid advancement of Large Language Models (LLMs), researchers have actively explored their potential in automating complex tasks within the domain of library evolution.

For deprecated APIs, Mahmud et al. [28] proposed GUPPY, the first LLM-based approach to automatically update Android deprecated API usages without requiring manual templates. Similarly, to prevent the introduction of outdated APIs during generation, Bai et al. [1] developed APILOT, which leverages real-time differential analysis to verify LLM outputs against a database of outdated APIs. Zhang et al.[46] enhanced Chain-of-Thought (CoT) by constructing task-specific reasoning patterns. In code synthesis, Li et al. [19] further advanced this by proposing Structured CoT (SCoT), which explicitly embeds programming logic (e.g., branches and loops) into the reasoning chain to constrain the model's output. Liu et al.[24] proposed ERA-CoT to capture entity relationships for better context understanding. Meanwhile, agent-based systems provide another promising approach for library evolution issues. Despite these advancements, fundamental challenges persist in applying LLMs to library evolution. Wang et al.[39] quantitatively revealed that LLMs frequently utilize deprecated APIs due to outdated training data. Liang et al.[21] confirmed that in the Retrieval-Augmented Generation (RAG), the knowledge cutoff significantly hampers performance.

Critically, existing solutions has a drawback that: current CoT approaches and multi-agent frameworks lack design for API compatibility issues. To address these limitations, our work introduces a specialized multi-agent framework. We design a CoT prompt mechanism to explicitly reason about incompatibility categories and orchestrate Input-Generator and Simulater agents to construct a closed-loop verification system.

C. LLM-powered Static Analysis

XXX

LLMDFA: Analyzing Dataflow in Code with Large Language Models

Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering

Llm-powered static binary taint analysis

VI. Conclusion

References

[1] W. Bai, K. Xuan, P. Huang, Q. Wu, J. Wen, J. Wu, and K. Lu, "Apilot: Navigating large language models to generate secure code by sidestepping outdated api pitfalls," *arXiv preprint arXiv:2409.16526*, 2024.

[2] A. Brito, L. Xavier, A. Hora, and M. T. Valente, "Apidiff: Detecting api breaking changes," in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2018, pp. 507–511.

[3] H. Cai, Z. Zhang, L. Li, and X. Fu, "A large-scale study of application incompatibilities in android," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 216–227.

[4] J. Chen, K. Li, Y. Chen, L. Wei, and Y. Liu, "Demystifying device-specific compatibility issues in android apps," in *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 525–537.

[5] L. Chen, F. Hassan, X. Wang, and L. Zhang, "Taming behavioral backward incompatibilities via cross-project testing and analysis," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 112–124.

[6] W. Cheng, W. Hu, and X. Ma, "Revisiting knowledge-based inference of python runtime environments: A realistic and adaptive approach," *IEEE Transactions on Software Engineering*, vol. 50, no. 2, pp. 258–279, 2023.

[7] Z. Dong, Y. Zhao, T. Liu, C. Wang, G. Xu, G. Xu, L. Zhang, and H. Wang, "Same app, different behaviors: Uncovering device-specific behaviors in android apps," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2099–2109.

[8] X. Du and J. Ma, "Aexpy: Detecting api breaking changes in python packages," in *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2022, pp. 470–481.

[9] D. He, L. Li, L. Wang, H. Zheng, G. Li, and J. Xue, "Understanding and detecting evolution-induced compatibility issues in android apps," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 167–177.

[10] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.

[11] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[12] H. Huang, L. Wei, Y. Liu, and S.-C. Cheung, "Understanding and detecting callback compatibility issues for android applications," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 532–542.

[13] D. Jayasuriya, V. Terragni, J. Dietrich, and K. Blincoe, "Understanding the impact of apis behavioral breaking changes on client applications," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1238–1261, 2024.

[14] W. Jin, J. Shang, J. Zheng, M. Sun, Z. Huang, M. Fan, and T. Liu, "The design smells breaking the boundary between android variants and aosp," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 599–599.

[15] D. Kong, J. Liu, L. Bao, and D. Lo, "Toward better comprehension of breaking changes in the npm ecosystem," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 1–23, 2025.

[16] D. Kong, J. Liu, C. Ni, D. Lo, and L. Bao, "More effective javascript breaking change detection via dynamic object relation graph," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 2340–2361, 2025.

[17] M. Lamothe, Y.-G. Guéhéneuc, and W. Shang, "A systematic review of api evolution literature," *ACM Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–36, 2021.

[18] C. Latappy, T. Degueule, J.-R. Falleri, R. Robbes, and L. Ochoa, "Roseau: Fast, accurate, source-based api breaking change analysis in java," *arXiv preprint arXiv:2507.17369*, 2025.

[19] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–23, 2025.

[20] L. Li, T. F. Bissyandé, H. Wang, and J. Klein, "Cid: Automating the detection of api-related compatibility issues in android apps," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2018, pp. 153–163.

[21] L. Liang, J. Gong, M. Liu, C. Wang, G. Ou, Y. Wang, X. Peng, and Z. Zheng, "Rustevo^ 2: An evolving benchmark for api evolution in llm-based rust code generation," *arXiv preprint arXiv:2503.16922*, 2025.

[22] P. Liu, Y. Zhao, H. Cai, M. Fazzini, J. Grundy, and L. Li, "Automatically detecting api-induced compatibility issues in android apps: a comparative analysis (replicability study)," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 617–628.

[23] P. Liu, Y. Zhao, M. Fazzini, H. Cai, J. Grundy, and L. Li, "Automatically detecting incompatible android apis," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 1–33, 2023.

[24] Y. Liu, X. Peng, T. Du, J. Yin, W. Liu, and X. Zhang, "Era-cot: improving chain-of-thought through entity relationship analysis," *arXiv preprint arXiv:2403.06932*, 2024.

[25] T. Mahmud, M. Che, J. Rouijel, M. Khan, and G. Yang, "Apicia: An api change impact analyzer for android apps," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 99–103.

[26] T. Mahmud, M. Che, and G. Yang, "Acid: an api compatibility issue detector for android apps," in *Proceedings of the ACM/IEEE 44th international conference on software engineering: Companion proceedings*, 2022, pp. 1–5.

[27] T. Mahmud, M. Che, and G. Yang, "An empirical study on compatibility issues in android api field evolution," *Information and Software Technology*, vol. 175, p. 107530, 2024.

[28] T. Mahmud, B. Duan, M. Che, A. Yasmin, A. H. Ngu, and G. Yang, "Automated update of android deprecated api usages with large language models," *arXiv preprint arXiv:2411.04387*, 2024.

[29] numpy.org, "Numpy reference," 2025. [Online]. Available: https://numpy.org/doc/1.20/numpy-ref.pdf

[30] python.org, "Pep 479 – change stopiteration handling inside generators," 2025. [Online]. Available: https://peps.python.org/pep-0479/

[31] readthedocs.io, "Python 3.12.0a0 documentation," 2025. [Online]. Available: https://pradyunsg-cpython-lutra-testing.readthedocs.io/en/latest/library/stdtypes.html

[32] K. Shen, K. Huang, B. Chen, and X. Peng, "Detecting incompatible third-party library apis in python based on static analysis," *Journal of Software (Chinese)*, vol. 36, no. 4, pp. 1435–1460, 2024.

[33] stackoverflow.com, "Is the order of a python dictionary guaranteed over iterations?" 2025. [Online]. Available: https://stackoverflow.com/questions/2053021/is-the-order-of-a-python-dictionary-guaranteed-over-iterations

[34] stackoverflow.com, "Numpy function type casting automatically or not," 2025. [Online]. Available: https://stackoverflow.com/questions/70079170/numpy-function-type-casting-automatically-or-not

[35] X. Sun, X. Chen, Y. Zhao, P. Liu, J. Grundy, and L. Li, "Mining android api usage to generate unit test cases for pinpointing compatibility issues," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[36] C.-C. Tsai, B. Jain, N. A. Abdul, and D. E. Porter, "A study of modern linux api usage and compatibility: What to support when you're supporting," in *Proceedings of the Eleventh European Conference on Computer Systems*, 2016, pp. 1–16.

[37] A. Vadlamani, R. Kalicheti, and S. Chimalakonda, "Apiscanner-towards automated detection of deprecated apis in python libraries," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2021, pp. 5–8.

[38] w3cub.com, "numpy.sum," 2025. [Online]. Available: https://docs.w3cub.com/numpy 1.17/generated/numpy.sum

[39] C. Wang, K. Huang, J. Zhang, Y. Feng, L. Zhang, Y. Liu, and X. Peng, "Llms meet library evolution: Evaluating deprecated api usage in llm-based code completion," *arXiv preprint arXiv:2406.09834*, 2024.

[40] J. Wang, G. Xiao, S. Zhang, H. Lei, Y. Liu, and Y. Sui, "Compatibility issues in deep learning systems: Problems and opportunities," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 476–488.

[41] L. Wei, Y. Liu, and S.-C. Cheung, "Taming android fragmentation: Characterizing and detecting compatibility issues for android apps," in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 226–237.

[42] L. Wei, Y. Liu, and S.-C. Cheung, "Pivot: learning api-device correlations to facilitate android compatibility issue detection," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 878–888.

[43] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying llm-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025.

[44] Y. Xie, Z. Jia, S. Li, Y. Wang, J. Ma, X. Li, H. Liu, Y. Fu, and X. Liao, "How to pet a two-headed snake? solving cross-repository compatibility issues with hera," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 694–705.

[45] S. Zhang, G. He, and G. Xiao, "Coding pitfalls: Demystifying the potential api compatibility risk of variadic parameters in python," in *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2024, pp. 105–106.

[46] Y. Zhang, X. Wang, L. Wu, and J. Wang, "Enhancing chain of thought prompting in large language models via reasoning patterns," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 985–25 993.

[47] Z. Zhang, H. Zhu, M. Wen, Y. Tao, Y. Liu, and Y. Xiong, "How do python framework apis evolve? an exploratory study," in *2020 ieee 27th international conference on software analysis, evolution and reengineering (saner)*. IEEE, 2020, pp. 81–92.

[48] Z. Zhao, B. Kou, M. Y. Ibrahim, M. Chen, and T. Zhang, "Knowledge-based version incompatibility detection for deep learning," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 708–719.

[49] C. Zhu, M. Zhang, X. Wu, X. Xu, and Y. Li, "Client-specific upgrade compatibility checking via knowledge-guided discovery," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–31, 2023.